Ministerul Educației și Cercetării a Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare Informatică și Microelectronică

RAPORT

la lucrarea de laborator nr. 3

la obiectul Planificarea și Estimarea Costurilor Proiectelor IT

Tema: *Crearea unei matrice de responsabilitate (RAM) pentru un proiect IT*

A efectuat: Botnarenco Ecaterina, TI-223
A verificat: Cojocaru Svetlana, asist. univ.

Chișinău 2025

Scopul lucrării:

Lucrarea de laborator constă în aplicarea metodologiilor de planificare specifice managementului proiectelor IT, prin elaborarea unei matrice de responsabilitate (RAM) care să stabilească în mod clar rolurile și nivelul de implicare al fiecărui membru al echipei în cadrul unui proiect software. Lucrarea urmărește structurarea sarcinilor în funcție de competențe, definirea responsabilităților individuale și identificarea relațiilor de colaborare necesare pentru realizarea obiectivelor. Prin utilizarea modelului RACI, se asigură o distribuție transparentă a responsabilităților, facilitând coordonarea activităților în derularea proiectului. Astfel, realizarea matricei contribuie la fundamentarea unui proces de management coerent, eficient și adaptat cerințelor proiectelor IT moderne.

Obiectivele lucrării:

- Dezvoltarea abilității de a organiza și aloca sarcini în cadrul unui proiect IT, în funcție de competențele și rolurile fiecărui membru al echipei.
- Învățarea modului de utilizare a matricei de responsabilitate (RAM) pentru a clarifica responsabilitățile și sarcinile fiecărei persoane implicate în proiect.
- Îmbunătățirea comunicării și colaborării în echipă, asigurând o distribuție echitabilă și transparentă a sarcinilor.
- Facilitarea gestionării proiectului printr-o alocare clară și eficientă a resurselor umane disponibile.

Introducere

Matricea de responsabilitate este un instrument de management pentru alocarea și clarificarea responsabilităților membrilor echipei în proiecte. Are drept scop asigurarea unei distribuții transparente a rolurilor, prevenind confuziile și suprapunerile de sarcini. Deseori este implementată folosind modelul RACI, acesta definind patru tipuri de implicare pentru fiecare sarcină.

Fiecare activitate din proiect este asociată cu unul sau mai multe roluri, folosind următoarele coduri:

- R – Responsible: persoana care execută efectiv sarcina.
- A – Accountable: persoana care răspunde final pentru rezultat; aprobă deciziile.
- C – Consulted: persoanele care oferă expertiză sau feedback.
- I – Informed: persoanele care trebuie doar informate despre progres sau rezultat.

Important: pentru fiecare activitate, există exact un singur A, dar pot exista mai multe roluri marcate cu R, C sau I.

De ce este importantă o matrice de responsabilitate în proiectele IT?

- Clarifică rolurile pentru fiecare etapă (analiză, dezvoltare, testare, livrare).
- Elimină ambiguitățile, astfel încât fiecare membru știe ce așteptări există.
- Previne conflictele privind asumarea sarcinilor.
- Optimizează comunicarea între echipe.
- Îmbunătățește planificarea și permite monitorizarea coerentă a progresului.
- Contribuie la controlul calității, fiindcă responsabilitățile sunt trasate corect.

În Anexa A este tabelul destinat etapei de analiză și planificare, el prezintă distribuția responsabilităților pentru activitățile fundamentale care stau la baza structurării proiectului. Managerul de proiect deține rolul central, asumând responsabilitatea finală pentru evaluarea domeniului, definirea cerințelor, stabilirea obiectivelor și construirea planului de execuție. Dezvoltatorul este implicat activ în colectarea cerințelor și analiza soluțiilor similare, întrucât experiența tehnică este esențială pentru conturarea funcționalităților reale ale aplicației. Arhitectul software contribuie ca expert consultat în definirea cerințelor și evaluarea fezabilității tehnice, în timp ce UI/UX și QA sunt implicați punctual, fiind informați sau consultați acolo unde este necesară expertiza lor specifică. Această structură evidențiază caracterul strategic al etapei și importanța colaborării interdisciplinare.

Tabelul din Anexa B este tabelul aferent etapei de design și prototipare reflectă concentrarea responsabilităților în zona arhitecturală și conceptuală a aplicației. Managerul de proiect asigură coordonarea generală, în timp ce arhitectul software preia rolul principal în proiectarea arhitecturii sistemului, a bazei de date și a mecanismelor de analiză semantică. Dezvoltatorul participă activ la definirea elementelor tehnice, iar specialistul UI/UX conduce etapa vizuală, responsabil pentru mockup-uri și validarea interfețelor. Specialistul QA intervine preponderent pentru evaluarea prototipurilor în raport cu cerințele funcționale și de utilizabilitate. Structura responsabilităților pune accent pe coerența proiectării și pe fundamentarea unei baze tehnice solide înainte de implementare.

Tabelul pentru etapa de dezvoltare evidențiază rolul dominant al dezvoltatorului, care preia responsabilitatea directă pentru implementarea fluxurilor esențiale ale aplicației: autentificare, încărcare de fișiere, algoritmi NLP, generarea rapoartelor și integrarea motorului semantic.

Arhitectul software este implicat constant în calitate de expert consultat, asigurând că deciziile tehnice rămân coerente cu arhitectura proiectată. Managerul de proiect monitorizează progresul și este informat asupra stadiului lucrărilor, iar specialistul QA contribuie prin verificarea tehnică a modulelor implementate. Această distribuție reflectă caracterul intens tehnic al etapei, unde majoritatea responsabilităților se concentrează asupra dezvoltatorului. Tabelul pentru etapa respectivă este afișat în Anexa C.

În Anexa D tabelul dedicat etapei de testare și asigurare a calității arată o redistribuire a responsabilităților, specialistul QA preluând rolul principal în verificarea funcțională, de performanță și de securitate a aplicației. Dezvoltatorul rămâne implicat ca executant în remedierea problemelor identificate, iar arhitectul software contribuie ca expert în interpretarea rezultatelor testelor, în special în zonele ce țin de performanța algoritmilor și securitatea infrastructurii. Managerul de proiect monitorizează derularea testelor, fiind informat asupra progresului și asupra eventualelor riscuri emergente. Structura acestui tabel subliniază separarea clară dintre dezvoltare și evaluare, esențială pentru obținerea unui produs stabil și sigur.

Tabelul destinat etapei de lansare și implementare evidențiază rolul central al managerului de proiect în coordonarea trecerii aplicației în mediul de producție. Acesta poate fi observat în Anexa E Dezvoltatorul este responsabil de configurarea serverului, migrarea bazei de date, publicarea versiunii web și configurarea accesului utilizatorilor. Arhitectul software oferă suport tehnic prin consultanță și verificarea conformității mediului cu arhitectura proiectată. Specialistul QA este implicat în validările finale pentru a asigura funcționarea corectă a aplicației în mediul live. Această etapă are un caracter operațional intens, iar matricea confirmă necesitatea colaborării strânse între rolurile tehnice și cele de management.

Tabelul din Anexa F este tabelul dedicat modelării aplicației punând în evidență rolul arhitectului software, care preia responsabilitatea pentru elaborarea modelelor conceptuale, logice și algoritmice ale sistemului. Activități precum modelarea bazei de date, definirea fluxurilor de date, conceptualizarea algoritmilor NLP și realizarea diagramelor UML sunt coordonate de arhitect, cu suport direct din partea dezvoltatorului. Managerul de proiect este informat despre evoluția acestei etape, iar QA și UI/UX sunt implicați doar punctual. Această structură subliniază importanța fundamentării teoretice și arhitecturale pentru un proiect care implică analiză semantică, algoritmi avansați și procese complexe de prelucrare a textului.

Tabelul aferent etapei de documentare indică rolul managerial esențial în supervizarea și consolidarea tuturor materialelor tehnice și operaționale ale proiectului. Arhitectul software se ocupă de documentarea arhitecturii și bazei de date, în timp ce dezvoltatorul contribuie la redactarea manualelor de utilizare și a documentației tehnice. Specialistul QA este responsabil pentru rapoartele de testare, iar UI/UX intervine în elaborarea ghidurilor destinate utilizatorilor. Această etapă garantează transferul complet al cunoștințelor, transparența tehnică și pregătirea proiectului pentru mentenanță și utilizare pe termen lung. Acest tabel este afișat în Anexa 7.

Concluzii

În urma realizării acestei lucrări, s-a demonstrat importanța utilizării unei matrice de responsabilitate în organizarea și coordonarea unui proiect IT. Structurarea sarcinilor în funcție de roluri și competențe a permis delimitarea clară a responsabilităților, eliminând posibile confuzii și suprapuneri în execuția activităților. Totodată, utilizarea modelului RACI a evidențiat modul în care fiecare membru contribuie la realizarea obiectivelor proiectului, fie prin execuție directă, fie prin supervizare, consultare sau informare.

Rezultatele obținute arată că o matrice de responsabilitate reprezintă un instrument valoros pentru îmbunătățirea comunicării și colaborării în cadrul proiectului, facilitând atât planificarea, cât și monitorizarea progresului. Distribuirea echilibrată a sarcinilor între Managerul de Proiect, Arhitectul Software, Dezvoltator, UI/UX și specialistul QA a reflectat necesitatea unui cadru organizatoric bine definit, capabil să susțină toate etapele dezvoltării software.

În ansamblu, lucrarea confirmă faptul că integrarea unei matrice RAM în planificarea proiectelor IT contribuie la creșterea coerenței proceselor, la optimizarea resurselor și la obținerea unui produs final realizat într-un mod structurat, predictibil și profesionist.

Anexa A

Sarcină	Manager Proiect	Arhitector Software	Dezvoltator	Specialist UI	Tester
1.1.1 Identificarea tipurilor de utilizatori	A	C	R	I	I
1.1.2 Colectarea cerințelor	A	C	R	C	I
1.2.1 Definirea rolurilor părților interesate	A	C	I	I	I
1.2.2 Stabilirea responsabilităților	A	C	I	I	I
1.3.1 Analiza platformelor TextGuard/JustDone	A	C	R	I	I
1.3.2 Identificarea funcționalităților cheie	A	C	R	I	I
1.4.1 Stabilirea scopului aplicației	A	C	I	I	I
1.4.2 Definirea obiectivelor tehnice	A	C	I	I	I
1.5.1 Cerințe funcționale	A	C	R	I	C
1.5.2 Cerințe nefuncționale	A	C	R	I	C
1.6.1 Identificarea resurselor tehnice	A	C	I	I	I
1.6.2 Estimarea costurilor	A	C	I	I	I
1.7.1 Calendarul activităților	A	C	I	I	I
1.7.2 Riscuri și plan de mitigare	A	C	I	I	I

Anexa B

Sarcină	Manager Proiect	Arhitector Software	Dezvoltator	Specialist UI	Tester
2.1.1 Definirea arhitecturii aplicației	A	A	C	I	I
2.1.2 Alegerea tehnologiilor	A	A	C	I	I
2.2.1 Crearea mockup-urilor	I	C	I	R	I
2.2.2 Validarea prototipurilor	I	C	I	R	C
2.3.1 Diagramelor ER	A	A	R	I	I
2.3.2 Definirea relațiilor	A	A	R	I	I
2.4.1 Alegerea modelelor NLP	A	A	R	I	I
2.4.2 Metodologia scorului	A	A	R	I	C
2.5.1 Autentificare pe roluri	A	A	R	I	C
2.5.2 Politici de securitate	A	A	R	I	C

Anexa C

Sarcină	Manager Proiect	Arhitector Software	Dezvoltator	Specialist UI	Tester
3.1.1 Autentificare pe roluri	I	C	R	I	C
3.1.2 Management sesiuni	I	C	R	I	C
3.2.1 Upload fișiere	I	C	R	I	C
3.2.2 Validare document	I	C	R	I	C
3.3.1 Preprocesare texte	I	C	R	I	C
3.3.2 Similaritate semantică	I	C	R	I	C
3.4.1 Highlight segmente similare	I	C	R	I	C
3.4.2 Scor final de similitudine	I	C	R	I	C
3.4.3 Export PDF/Excel	I	C	R	I	C
3.5.1 Integrarea modelului NLP	I	A	R	I	C
3.5.2 Optimizarea performanței	I	A	R	I	C

Anexa D

Sarcină	Manager Proiect	Arhitector Software	Dezvoltator	Specialist UI	Tester
4.1.1 Flux complet	I	C	R	I	A
4.1.2 Autentificare	I	C	R	I	A
4.1.3 Generarea rapoartelor	I	C	R	I	A
4.2.1 Documente mari	I	C	C	I	A
4.2.2 Utilizare intensă	I	C	C	I	A
4.3.1 Criptare date	I	A	C	I	R
4.3.2 Acces neautorizat	I	A	C	I	R

Anexa E

Sarcină	Manager Proiect	Arhitector Software	Dezvoltator	Specialist UI	Tester
5.1.1 Configurarea serverului	A	C	R	I	C
5.1.2 Migrare DB + teste finale	A	C	R	I	C
5.2.1 Publicare versiune web	A	C	R	I	C
5.2.2 Configurare acces utilizatori	A	C	R	I	C
5.3.1 Monitorizare erori	A	I	R	I	C
5.3.2 Feedback și corecții	A	I	R	I	C

Anexa F

Sarcină	Manager Proiect	Arhitector Software	Dezvoltator	Specialist UI	Tester
6.1.1 Stabilirea entităților	I	A	R	I	I
6.1.2 Definirea relațiilor	I	A	R	I	I
6.2.1 Diagrame activitate	I	A	R	I	I
6.2.2 Responsabilitățile componentelor	I	A	R	I	I
6.3.1 Descriere matematică	I	A	R	I	I
6.3.2 UML pentru scor	I	A	R	I	I
6.4.1 UML Clase/Secvență	I	A	R	I	I
6.4.2 Interacțiuni module	I	A	R	I	I

Anexa G

Sarcină	Manager Proiect	Arhitector Software	Dezvoltator	Specialist UI	Tester
7.1.1 Backend	A	R	C	I	I
7.1.2 Bază de date	A	R	C	I	I
7.2.1 Manual student	A	C	R	R	I
7.2.2 Manual profesor/admin	A	C	R	R	I
7.3.1 Raport teste	A	C	C	I	R
7.3.2 Raport erori	A	C	C	I	R
7.4.1 Plan suport/mentenanță	A	C	R	I	I
7.4.2 Jurnal versiuni	A	C	R	I	I

BIBLIOGRAFIE